

WebSocket

Lorenzo Marconi

Tecnologie e Sistemi Web

Corso di laurea in Ingegneria Informatica e Automatica

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica



SAPIENZA
UNIVERSITÀ DI ROMA

Anno accademico 2025/2026

- 1 WebSocket lato client
- 2 WebSocket in PHP
- 3 WebSocket in Node.js

Che cos'è una WebSocket dal punto di vista del client

- La WebSocket è un canale persistente tra browser e endpoint remoto, utile quando serve scambio di messaggi in tempo reale
- Dal client, il concetto chiave è che la connessione non è più di tipo “richiesta-risposta”: dopo l’handshake, il browser mantiene un socket aperto
- In JavaScript l’API nativa è WebSocket: non richiede librerie esterne per l’uso base nel browser
- Il client riceve eventi asincroni: open, message, error, close
- Tipico uso: chat, dashboard live, notifiche, tracking di stati, giochi multiplayer leggeri, collaborazione in tempo reale

Intuitivamente...

HTTP è ottimo per scambiare informazioni “ogni tanto”; WebSocket è ottimo per restare collegati e ricevere eventi appena arrivano

- Si crea con `new WebSocket(url)` usando `ws://` oppure `wss://`
- La connessione passa per stati standard: `CONNECTING`, `OPEN`, `CLOSING`, `CLOSED`
- Il client non “attende” in modo bloccante: reagisce a eventi e callback
- `onopen` indica che il canale è pronto per inviare dati
- `onclose` serve per pulizia, UI e, spesso, per avviare una strategia di riconnessione

Implementazione essenziale in JavaScript

```
1  const ws = new WebSocket('ws://example.com/socket');
2
3  ws.addEventListener('open', () => {
4      console.log('Connessione aperta');
5      ws.send(JSON.stringify({ type: 'hello', userId: 123 }));
6  });
7
8  ws.addEventListener('message', (event) => {
9      const data = JSON.parse(event.data);
10     console.log('Messaggio ricevuto:', data.msg);
11 });
12
13 ws.addEventListener('error', () => {
14     console.warn('Errore di comunicazione');
15 });
16
17 ws.addEventListener('close', (event) => {
18     console.log('Connessione chiusa', event.code, event.reason);
19 });
```

- `send()` invia stringhe o binari; in pratica spesso si usa JSON
- `event.data` contiene il payload ricevuto

Overview

- 1 WebSocket lato client
- 2 WebSocket in PHP**
- 3 WebSocket in Node.js

- Nel PHP tradizionale la logica è spesso *request/response*: il server risponde ad una chiamata HTTP e termina
- Con le WebSocket la connessione resta *aperta*
- Il server può ricevere messaggi in qualunque momento e rispondere senza attendere nuove richieste HTTP
- Il codice diventa guidato da eventi (es. connessione, messaggio, chiusura, errore)

- **Composer** è un *gestore di dipendenze* (o *package manager*, “gestore di pacchetti”) per PHP
 - Analogo di `npm` per Node.js
- Permette di installare librerie esterne in modo ripetibile e dichiarativo
- Setup scaricabile al sito *getcomposer.org*

Dopo aver scaricato e installato Composer, possono essere usati i seguenti comandi:

- **composer require** nome-pacchetto installa il pacchetto "nome-pacchetto" nel seguente modo:
 - crea (o aggiorna) il file `composer.json` che elenca le dipendenze installate
 - crea (o aggiorna) il file `composer.lock` che riassume tutti i metadati delle dipendenze
 - crea (o aggiorna) la cartella `vendor/` con tutte le dipendenze
 - crea (se assente) il file `vendor/autoload.php`: sarà poi possibile includere questo file anziché le dipendenze una per una
- **composer install**: installa esattamente ciò che è nel `composer.lock`
- **composer update**: aggiorna `composer.lock` con versioni più recenti compatibili

- **Ratchet** è una libreria PHP per costruire server WebSocket
- Installabile tramite Composer:

```
composer require cboden/ratchet
```

- Si basa su un modello *event-driven*: il codice reagisce agli eventi della connessione
- Fornisce componenti già pronti per accettare client, ricevere messaggi e inviare risposte

- Un **namespace** separa classi con lo stesso nome ma provenienti da contesti diversi
- Il backslash separa “sottospazi” di nomi, e il backslash iniziale indica il namespace globale
- La keyword **use** serve a importare un *qualifier* definito all'interno di un namespace
 - Evita di scrivere continuamente il “percorso” completo
 - Rende il codice più leggibile, soprattutto quando si usano classi di librerie esterne

```
1 use Ratchet\MessageComponentInterface;  
2 use Ratchet\ConnectionInterface;
```

- Dopo l'import, si può usare solo il nome breve della classe

- L'interfaccia `MessageComponentInterface` è il “contratto” che un componente Ratchet deve rispettare per gestire gli eventi `WebSocket`
- Quattro metodi fondamentali:
 - `onOpen`
 - `onMessage`
 - `onClose`
 - `onError`
- Il server chiama questi metodi quando succede qualcosa sulla connessione

onOpen: quando un client si collega

- Viene chiamato nel momento in cui una nuova connessione WebSocket è aperta
- Qui si registrano spesso le connessioni attive
- È il punto giusto per inizializzare dati legati al client: ID, stato, timestamp, risorse condivise

Esempio (minimale)

```
1 public function onOpen(ConnectionInterface $conn) {  
2     $this->clients->attach($conn);  
3 }
```

In parole semplici: “è arrivato un nuovo utente, lo memorizzo”

onMessage: quando arriva un messaggio

- Viene chiamato ogni volta che il server riceve un messaggio da un client
- Gli argomenti della funzione indicano chi ha inviato il messaggio e qual è il contenuto ricevuto
- Qui si implementa la logica applicativa: chat, notifiche, comandi, broadcast, ...

Esempio

Inoltrare il messaggio a tutti i client

```
1 public function onMessage(ConnectionInterface $from, $msg) {  
2     foreach ($this->clients as $client) {  
3         $client->send($msg);  
4     }  
5 }
```

onClose: quando il client si disconnette

- Viene chiamato quando la connessione viene chiusa
- Serve a liberare risorse e rimuovere il client dalle strutture dati interne
- È importante per evitare memoria occupata inutilmente e dati “fantasma”

Esempio

```
1 public function onClose(ConnectionInterface $conn) {  
2     $this->clients->detach($conn);  
3 }
```

In pratica: “questo client non c’è più, lo tolgo dall’elenco”

- Viene chiamato quando la comunicazione genera un problema
- Permette di loggare l'errore, chiudere la connessione e mantenere il server stabile
- Una buona gestione degli errori evita che un singolo client comprometta tutto il servizio

Esempio

```
1 public function onError(ConnectionInterface $conn, \Exception  
2     $e) {  
3     $conn->close();  
    }
```

Spesso si aggiunge anche un log dell'eccezione per scopi di debug

- 1 WebSocket lato client
- 2 WebSocket in PHP
- 3 **WebSocket in Node.js**
 - La libreria `ws`
 - La libreria `socket.io`

- In Node.js il codice gira in modo *event-driven*: il server reagisce a eventi invece di bloccare l'esecuzione
- Le WebSocket si inseriscono bene in questo modello perché anche la comunicazione è guidata da eventi
- Il server mantiene connessioni aperte e può inviare messaggi in qualunque momento
- Per l'uso base si utilizza spesso la libreria `ws`

- La libreria `ws` fornisce un server WebSocket semplice e leggero
- Installazione:

```
npm install ws
```

- Dopo l'installazione si può creare un server WebSocket in poche righe

Server WebSocket minimale in Node.js

```
1 const WebSocket = require('ws');
2
3 const wsServer = new WebSocket.Server({ port: 9000 });
4
5 wsServer.on('connection', (socket) => {
6     socket.send(JSON.stringify({ type: 'welcome' }));
7
8     socket.on('message', (message) => {
9         console.log('Ricevuto:', message.toString());
10        socket.send(message); // Mandiamo lo stesso messaggio indietro al client
11    });
12
13    socket.on('close', () => {
14        console.log('Client disconnesso');
15    });
16
17    socket.on('error', (err) => {
18        console.error('Errore:', err);
19    });
20 });
```

- `connection`: arriva un nuovo client
- `message`: il client invia un messaggio
- `close`: la connessione viene chiusa
- `error`: si verifica un problema di comunicazione

Nella maggior parte dei casi, allo stesso tempo, si vuole gestire anche un server HTTP. In tal caso, non è necessario creare un server WebSocket del tutto disaccoppiato, ma si può “dirottare” il traffico HTTP

Durante la creazione:

```
new WebSocket.Server({ port: 9000 });
```

in alternativa al numero di porta, tra le opzioni passate al costruttore `WebSocket.Server()` si può specificare un oggetto `http.Server` (o `https.Server`) già istanziato (v. slide successiva)

Unico server HTTP + WebSocket (esempio)

```
1  const http = require('http');
2  const WebSocket = require('ws');
3
4  const httpServer = http.createServer((req, res) => {
5      /* ... */
6  });
7
8  const wsServer = new WebSocket.Server({ server: httpServer });
9  wsServer.on('connection', (ws) => {
10     /* ... */
11 });
12
13 httpServer.listen(9000, () => {
14     console.log('Server HTTP + WebSocket sulla porta 9000');
15 });
```

Unico server HTTP + WebSocket (esempio con Express.js)

```
1 const express = require('express');
2 const http = require('http');
3 const WebSocket = require('ws');
4
5 const app = express();
6
7 // Routing HTTP normale
8 app.get('/', (req, res) => {
9     /* ... */
10 });
11
12 const httpServer = http.createServer(app);
13 const wsServer = new WebSocket.Server({ server: httpServer });
14 wsServer.on('connection', (ws, req) => {
15     /* ... */
16 });
17
18 httpServer.listen(9000, () => {
19     console.log('HTTP + WS su porta 9000');
20 });
```


- **socket.io** è una libreria per comunicazione in tempo reale tra client e server
- Espone un'API basata su eventi, molto comoda per applicazioni interattive
- Non si limita alla WebSocket pura: aggiunge funzionalità pratiche come riconnessione automatica e *rooms*
- Viene spesso usata quando si vuole una soluzione più “alta livello” rispetto alle WebSocket nude

Differenza intuitiva rispetto alle WebSocket “pure”

- Con `WebSocket` si lavora più vicino al protocollo e si gestiscono manualmente molti dettagli
- Con `socket.io` si ragiona in termini di eventi applicativi (tramite le funzioni `emit` e `on`)
- La libreria aggiunge comodità operative, ad esempio:
 - riconnessione automatica
 - supporto a *rooms*
 - broadcasting più semplice

- Installazione lato server:

```
npm install socket.io
```

- Sul client si usa lo script della libreria o il relativo pacchetto
- Il modello resta semplice: il server definisce eventi, il client li ascolta e viceversa

Server socket.io minimale

```
1  const { Server } = require('socket.io');
2
3  const io = new Server(9000);
4
5  io.on('connection', (socket) => {
6      console.log('Utente connesso');
7
8      socket.emit('welcome', { message: 'Benvenuto!' });
9
10     socket.on('chat', (msg) => {
11         io.emit('chat', msg);
12     });
13
14     socket.on('disconnect', () => {
15         console.log('Utente disconnesso');
16     });
17 });
```

L'uso combinato con `http/Express.js` è analogo a quanto visto per la libreria `ws` (es. slides 23-24), ovvero, al posto di `new Server(9000)` è possibile chiamare `new Server(httpServer)`

Client socket.io minimale

```
1  const socket = io('http://localhost:9000');
2
3  socket.on('welcome', (data) => {
4      console.log(data.message);
5  });
6
7  socket.on('chat', (msg) => {
8      console.log('Messaggio:', msg);
9  });
10
11 socket.emit('chat', 'Ciao a tutti!');
```

- `emit` invia un evento con un nome e un payload
- `on` registra un gestore per ricevere quell'evento
- Il codice diventa molto leggibile perché ogni messaggio ha un “nome” applicativo

- Una *room* è un gruppo logico di socket
- Permette di inviare messaggi solo a un sottoinsieme di client
- È utile per chat di gruppo, stanze di gioco, canali tematici, ...
- Room di default: `socket.id` (ID univoco della socket stessa)

- `io.on("connection", socket => { ... })`
- `io.emit("evento-custom", data)`
- `io.to("stanza").emit("evento", data)`
- `socket.on("disconnect", () => { ... })`
- `socket.on("evento-custom", (data) => { ... })`
- `socket.emit("evento-custom", data)`
- `socket.broadcast.emit("evento-custom", data)`
- `socket.to("stanza").emit("evento", data)`
- `socket.join("stanza")`
- `socket.leave("stanza")`

- `socket.on("evento-custom", (data) => ... )`
- `socket.emit("evento-custom", data)`
- `socket.disconnect()`